

auto-autoformalization

Freek Wiedijk

Institute for Computing and Information Sciences
Radboud University Nijmegen

AITP 2026 – Aussois, 2 August 2026

opinions on AI

Steve Jobs's opinion on taste

The only problem with Microsoft is they just have no taste. They have absolutely no taste. And what that means is, I don't mean that in a small way, I mean it in a big way. In the sense that they don't think of original ideas, and they don't bring much culture into their product.

And so I guess I am saddened, not by Microsoft's success. I have no problem with their success. They've earned their success – for the most part. I have a problem with the fact that they just really make third rate products.

opinions on AI

Steve Jobs's opinion on taste, ported to LLMs

The only problem with the **large language models** is they just have **no taste**. They have absolutely no taste. And what that means is, I don't mean that in a small way, I mean it in a big way. In the sense that they don't think of original ideas, and they don't bring much culture into their product.

And so I guess I am saddened, not by the **large language models'** success. I have no problem with their success. They've earned their success – for the most part. I have a problem with the fact that they just really make **third rate** products.

twelve angry lines

- ▶ AI is built on stolen books, stolen art, stolen code
- ▶ AI takes your job, and your kid's first one
- ▶ AI drinks our water and burns a planet we can't replace
- ▶ AI drowns the world in fakes and slop
- ▶ AI hands the world to five companies
- ▶ AI is biased, but they sell it as neutral
- ▶ AI is a bubble, and we pay when it bursts
- ▶ AI thinks for us until we can't
- ▶ AI teaches us to prefer machines to people
- ▶ AI weaponizes anyone who just asks
- ▶ AI gets obnoxiously pushed in your face everywhere
- ▶ AI will outgrow us, escape us, end us

twelve angry lines

- ▶ AI is built on stolen books, stolen art, stolen code
- ▶ AI takes your job, and your kid's first one
- ▶ AI drinks our water and burns a planet we can't replace
- ▶ AI drowns the world in fakes and slop
- ▶ AI hands the world to five companies
- ▶ AI is biased, but they sell it as neutral
- ▶ AI is a bubble, and we pay when it bursts
- ▶ AI thinks for us until we can't
- ▶ AI teaches us to prefer machines to people
- ▶ AI weaponizes anyone who just asks
- ▶ AI gets obnoxiously pushed in your face everywhere
- ▶ AI will outgrow us, escape us, end us – the AI apocalypse

- ▶ AI is built on stolen books, stolen art, stolen code
- ▶ AI takes your job, and your kid's first one
- ▶ AI drinks our water and burns a planet we can't replace
- ▶ AI drowns the world in fakes and slop
- ▶ AI hands the world to five companies
- ▶ AI is biased, but they sell it as neutral
- ▶ AI is a bubble, and we pay when it bursts
- ▶ AI thinks for us until we can't
- ▶ AI teaches us to prefer machines to people
- ▶ AI weaponizes anyone who just asks
- ▶ AI gets obnoxiously pushed in your face everywhere
- ▶ AI will outgrow us, escape us, end us – the AI apocalypse

autoformalization

... is a solved problem

`/goal` do not stop until you have fully formalized `scan/textbook.pdf` as a `Lean formalization`, without using any axioms or sorries or excuses or hidden assumptions, and where the formalization fully matches the text without omitting or changing or inventing anything, and with everything in the formalization ready to be submitted as PRs for the `Mathlib library`

autoformalization

... is a solved problem

`/goal` do not stop until you have fully formalized `scan/textbook.pdf` as an Isabelle/HOL formalization, without using any axioms or sorries or excuses or hidden assumptions, and where the formalization fully matches the text without omitting or changing or inventing anything, and with the formalization ready to be submitted to the [Archive of Formal Proofs](#)

autoformalization

... is a solved problem?

`/goal` do not stop until you have fully formalized `scan/textbook.pdf` as an Isabelle/HOL formalization, without using any axioms or sorries or excuses or hidden assumptions, and where the formalization fully matches the text without omitting or changing or inventing anything, and with the formalization ready to be submitted to the [Archive of Formal Proofs](#)

autoformalization

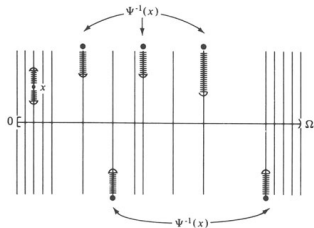
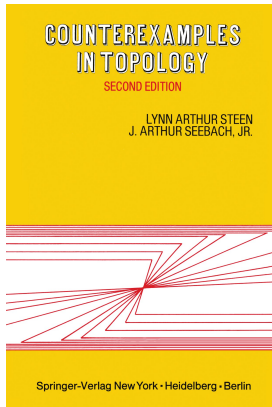
... will be a solved problem very soon!

`/goal` do not stop until you have fully formalized `scan/textbook.pdf` as an Isabelle/HOL formalization, without using any axioms or sorries or excuses or hidden assumptions, and where the formalization fully matches the text without omitting or changing or inventing anything, and with the formalization ready to be submitted to the [Archive of Formal Proofs](#)

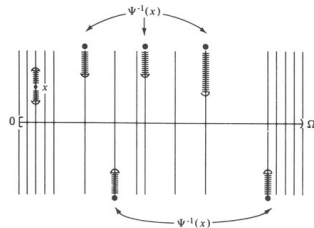
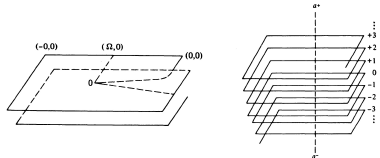
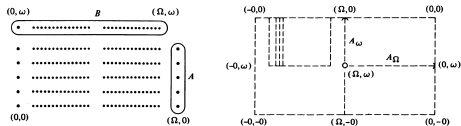
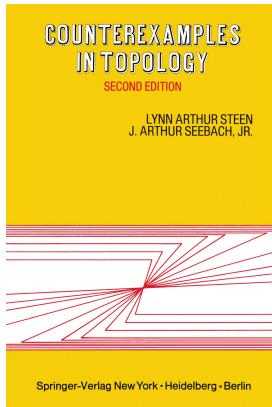
no extra AITP science will be needed, all you will need is tokens

🎵 all you need is tokens 🎵

an example: Hewett's condensed corkscrew in Lean



an example: Hewett's condensed corkscrew in Lean



auditing it

```
namespace SteenSeebach
```

```
-- Blueprint 1.1
```

```
-- "T_3 axiom: If A is a closed set and b is a point not in A, there  
-- exist disjoint open sets O_A and O_b containing A and b  
-- respectively." (S&S p. 11) = Mathlib.RegularSpace
```

```
abbrev T3 (X : Type*) [TopologicalSpace X] := RegularSpace X
```

```
-- Blueprint 1.2
```

```
-- "A space X is said ... to be regular if and only if it is both a T_0  
-- and a T_3 space" (S&S p. 12) = Mathlib.T3Space
```

```
abbrev Regular (X : Type*) [TopologicalSpace X] := T3Space X
```

```
instance CondensedCorkscrew.SS_regular : SteenSeebach.Regular  
  CondensedCorkscrew := by
```

```
...
```

```
theorem CondensedCorkscrew.not_SS_complReg :
```

```
  ¬ SteenSeebach.CompletelyRegular CondensedCorkscrew
```

```
:= by
```

```
...
```

auditing it

```
namespace SteenSeebach
```

```
-- Blueprint 1.1
```

```
-- "T_3 axiom: If A is a closed set and b is a point not in A, there  
-- exist disjoint open sets O_A and O_b containing A and b  
-- respectively." (S&S p. 11) = Mathlib.RegularSpace
```

```
abbrev T3 (X : Type*) [TopologicalSpace X] := RegularSpace X
```

```
-- Blueprint 1.2
```

```
-- "A space X is said ... to be regular if and only if it is both a T_0  
-- and a T_3 space" (S&S p. 12) = Mathlib.T3Space
```

```
abbrev Regular (X : Type*) [TopologicalSpace X] := T3Space X
```

```
instance CondensedCorkscrew.SS_regular : SteenSeebach.Regular  
  CondensedCorkscrew := by
```

```
...
```

```
theorem CondensedCorkscrew.not_SS_complReg :
```

```
  ¬ SteenSeebach.CompletelyRegular CondensedCorkscrew
```

```
:= by
```

```
...
```

should the library force the same conventions on everyone?

auditing it

```
namespace SteenSeebach
```

```
-- Blueprint 1.1
```

```
-- "T3 axiom: If A is a closed set and b is a point
```

```
-- exist disjoint open sets OA and Ob containing
```

```
-- respectively." (S&S p. 11) = Mathlib.RegularSpace
```

```
abbrev T3 (X : Type*) [TopologicalSpace X] := RegularSpace X
```

```
-- Blueprint 1.2
```

```
-- "A space X is said ... to be regular if and only if
```

```
-- and a T3 space" (S&S p. 12) = Mathlib.T3Space
```

```
abbrev Regular (X : Type*) [TopologicalSpace X] := T3Space X
```

```
instance CondensedCorkscrew.SS_regular : SteenSeebach.Regular  
  CondensedCorkscrew := by
```

```
...
```

```
theorem CondensedCorkscrew.not_SS_complReg :
```

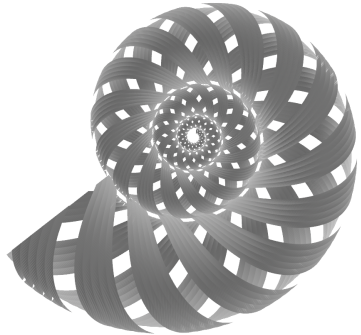
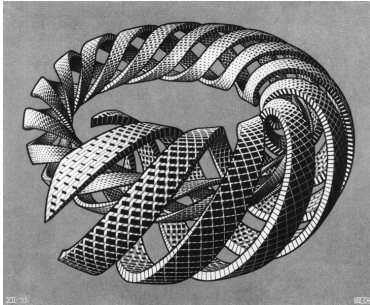
```
  ¬ SteenSeebach.CompletelyRegular CondensedCorkscrew
```

```
:= by
```

```
...
```

one Mathlib to bring them all, and in the darkness bind them

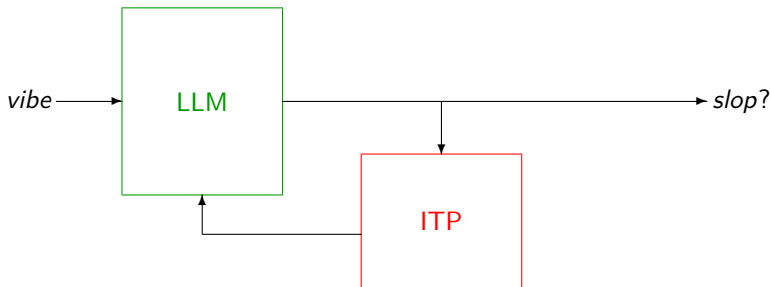
but vibe mathematics is not always correct



the story of vibe and slop: how to keep LLMs on track?

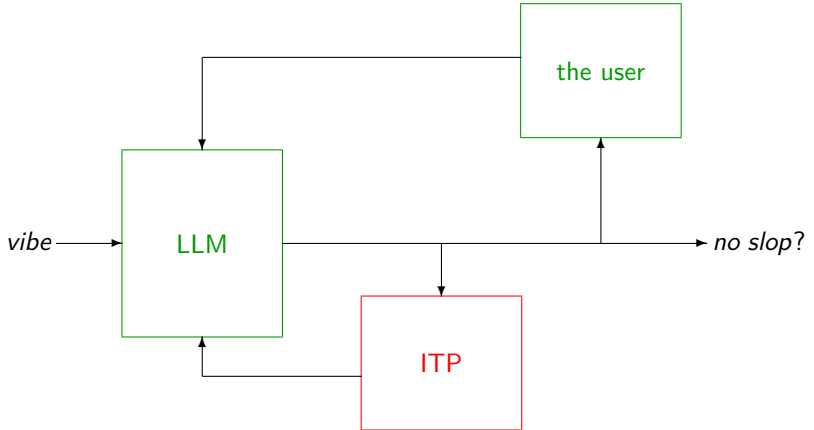


the story of vibe and slop: how to keep LLMs on track?



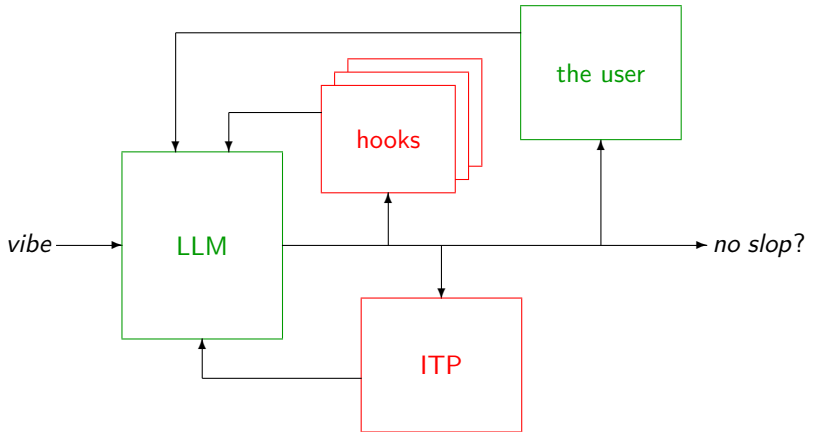
combine the strengths of **neural** and **symbolic**

the story of vibe and slop: how to keep LLMs on track?



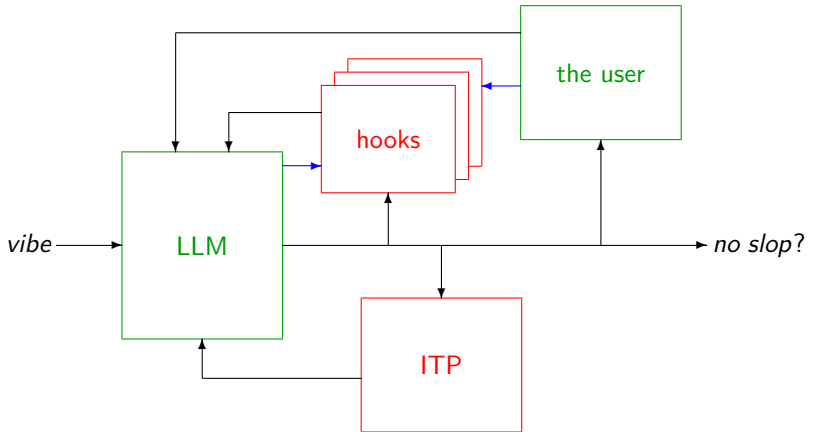
combine the strengths of **neural** and **symbolic**

the story of vibe and slop: how to keep LLMs on track?



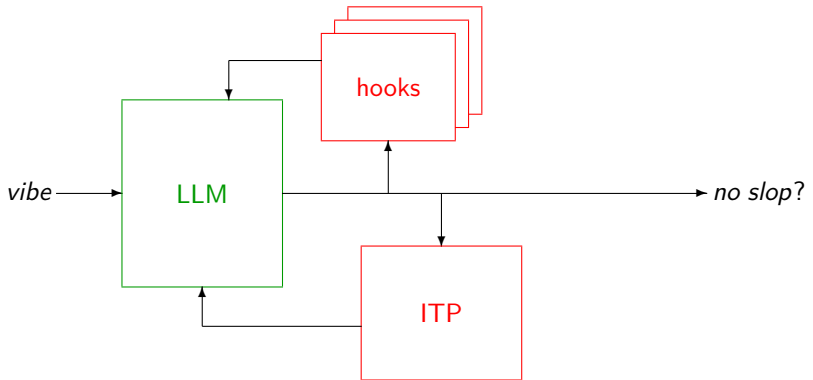
combine the strengths of **neural** and **symbolic**

the story of vibe and slop: how to keep LLMs on track?



combine the strengths of **neural** and **symbolic**

the story of vibe and slop: how to keep LLMs on track?



combine the strengths of **neural** and **symbolic**

the project

goals

autoformalization, but also:

- ▶ no human help allowed
auto-autoformalization
- ▶ properly match the source
 - ▶ no hidden cheats
 - ▶ no omissions
 - ▶ no changes
 - ▶ no inventions
- ▶ fit well with the library that is used
 - ▶ no duplication of existing library material
 - ▶ correct convention bridging
 - ▶ ready to be integrated into the library

the project

goals

autoformalization, but also:

- ▶ no human help allowed – only then will it scale!
auto-autoformalization
- ▶ properly match the source
 - ▶ no hidden cheats
 - ▶ no omissions
 - ▶ no changes
 - ▶ no inventions
- ▶ fit well with the library that is used
 - ▶ no duplication of existing library material
 - ▶ correct convention bridging
 - ▶ ready to be integrated into the library

formal proof sketches

Thm 4-1.

Let p be a prime

and $q = 2 \cdot 3 \cdot 5 \cdots p + 1$, one more than the product of all primes up to p .

Then q has a prime divisor d

with $p < d \leq q$.

Proof.

Since q leaves remainder 1 on division by each of $2, 3, 5, \dots, p$,

none of these primes divides q .

Hence q is either prime,

in which case

q itself is such a divisor (with $d = q$),

or

it has a prime divisor strictly between p and q .

In either case q has a prime divisor d with $p < d \leq q$.

theorem Thm_4_1:

```
fixes p :: nat
assumes "prime p"

defines "q ≡ (∏ r | prime r ∧ r ≤ p. r) + 1"

shows "∃d. prime d ∧ d dvd q
      ∧ p < d ∧ d ≤ q"
(is "∃d. ?such_divisor d")
```

proof -

```
{
  fix r :: nat
  assume "prime r ∧ r ≤ p"
  have "q mod r = 1"
    sorry1

  have "¬ r dvd q"
    sorry2
}

then have "prime q
  ∧
  ?such_divisor q
  ∨
  (∃d. prime d ∧ d dvd q ∧ p < d ∧ d < q)"
  sorry3

then show "∃d. prime d ∧ d dvd q ∧ p < d ∧ d ≤ q"
  sorry4
```

□ oed

the approach: a pipeline

scanned mathematical source
(article/textbook)

⋈

L^AT_EX clone of the source

⊃

massaged into a blueprint*

⋈

formal proof sketch of the blueprint

⊃

full formalization

⋈ as close as possible

⊃ should not omit anything

* *not* a Lean blueprint

scanned mathematical source
(article/textbook)

⋈

L^AT_EX clone of the source

⊃

massaged into a blueprint*

⋈

formal proof sketch of the blueprint

⊃

full formalization

⋈ as close as possible

⊃ should not omit anything

* *not* a Lean blueprint

the LLM does not get it

Thm II.14-7. (Theorem 14 · 1, Corollary.) If disjoint F_1, F_2 are compact, then $\rho(F_1, F_2) > 0$; this holds already when F_1 is compact and F_2 merely closed. If neither is compact, $\rho(F_1, F_2)$ may be 0 though both are closed (e.g. the ξ_1 -axis and $[\xi_2 = e^{-\xi_1}]$ in R^2).

Proof. If both are compact, II.14-6 gives $a_1 \in F_1, a_2 \in F_2$ with $\rho(F_1, F_2) = \rho(a_1, a_2) > 0$ (as $a_1 \neq a_2$). If only F_1 is compact and F_2 closed, take x_n, y_n with $\rho(x_n, y_n) \rightarrow \rho(F_1, F_2)$ and $x_{m_r} \rightarrow a_1 \in F_1$; then $\rho(F_1, F_2) = \lim \rho(x_{m_r}, y_{m_r}) = \lim \rho(a_1, y_{m_r}) \geq \rho(a_1, F_2) > 0$, since $a_1 \notin F_2$ and F_2 is closed.

□

Thm II.14-8. (Theorem 14 · 2.) E is compact if and only if every infinite subset of E has at least one limit-point in E .

Proof. Necessity. An infinite subset X contains distinct points $x_1, x_2, \dots$; a subsequence $x_{n_r} \rightarrow a \in E$, and by II.12-8 a is a limit-point of $\{x_{n_r}\}$, a fortiori of X . *Sufficiency.* Let (x_n) be any sequence in E . If some value recurs infinitely often it gives a constant convergent subsequence. Otherwise $\{x_n\}$ is an infinite subset with a limit-point $a \in E$; choose $x_{n_1} \in U(a, 1)$ and inductively $n_r > n_{r-1}$ least with $x_{n_r} \in U(a, 1/r)$ (possible since $\{x_n\} \cap U(a, 1/r)$ is infinite). Then $x_{n_r} \rightarrow a \in E$.

□

Thm II.14-9. (Theorem 14 · 3; Bolzano-Weierstrass.) A set

of points in R^p is compact if and only if it is bounded and closed. *Proof. Necessity* is II.14-2 and II.14-5. *Sufficiency* uses only that a bounded monotone sequence of reals converges. (1) $p = 1$. Let $F \subseteq \langle \alpha_1, \beta_1 \rangle$ and (ξ_n) a sequence in F ; put $\xi_{n_1} = \xi_1$. Bisect repeatedly: if $\langle \alpha_k, \beta_k \rangle$ contains ξ_n for infinitely many n , one half does too — call it $\langle \alpha_{k+1}, \beta_{k+1} \rangle$ and choose $\xi_{n_{k+1}}$ in it with $n_{k+1} > n_k$. Then (α_k) increases, (β_k) decreases, both bounded, so both converge; as $|\alpha_k - \beta_k| = 2^{-k+1}|\alpha_1 - \beta_1| \rightarrow 0$, to a common limit λ . Every $(\lambda - \varepsilon, \lambda + \varepsilon)$ contains $\langle \alpha_r, \beta_r \rangle$, hence ξ_{n_r} , for large r ; so $\xi_{n_r} \rightarrow \lambda$ and $\lambda \in F$ as F is closed. (2) $n > 1$. If F is bounded

```
lemma (in metric) Thm_II_14_7:
  assumes "F1 ⊆ S" and "F1 ≠ {}" and "mcompact F1"
    and "mclosed F2" and "F2 ≠ {}" and "F1 \<inter> F2 = {}"
  shows "dist_sets F1 F2 > 0"
```

sorry₁

```
lemma (in metric) Thm_II_14_8:
  assumes "E ⊆ S"
  shows "mcompact E ⟷ (∀X. X ⊆ E ∧ infinite X ⟶ (∃a∈E.
    a ∈ deriv X))"
```

sorry₁

a marked up proof text before LLMs

THEOREM 0.1. *Let n be an integer greater than or equal to 3. If integers X , Y , and Z satisfy the equation*

$$(0.1) \quad X^n + Y^n = Z^n,$$

then at least one of X , Y , and Z must be 0.

A flow diagram of the proof can be drawn as follows:

$$(0.2) \quad \begin{array}{ll} \text{(a solution of (0.1))} & \implies \text{(an elliptic curve)} \\ & \implies \text{(a modular form)} \\ & \implies \text{(contradiction)} \end{array}$$

If we try to explain the meaning of this diagram in a few sentences, it goes as follows. Assume there exists a nontrivial solution to the equation (0.1), and we would like to derive a contradiction. To this end, we define an elliptic curve using such a solution. We then show that such an elliptic curve is closely associated with a modular form with certain properties. Finally, we derive a contradiction by showing that such a modular form could not exist.

In this chapter we give a further explanation of the above diagram. As we can easily see, elliptic curves and modular forms play leading roles in the proof. By following the outline of the proof, the reader should familiarize her/himself with these two subjects. We indicate

a marked up proof text before LLMs

$\langle \text{label} \rangle$. Let $\langle \text{var} \rangle$ be an $\langle \text{type} \rangle$. If $\langle \text{type} \rangle \langle \text{var} \rangle$, $\langle \text{var} \rangle$, and $\langle \text{var} \rangle$ satisfy the equation

$\langle \text{label} \rangle$ $\langle \text{statement} \rangle$,

then $\langle \text{statement} \rangle$.

A $\langle \text{label} \rangle$ can be drawn as follows:

	$(a \langle \text{type} \rangle)$	$\implies$	$(an \langle \text{type} \rangle)$
$\langle \text{label} \rangle$		$\implies$	$(a \langle \text{type} \rangle)$
		$\implies$	(contradiction)

If we try to explain the meaning of $\langle \text{ref} \rangle$ in a few sentences, it goes as follows. Assume $\langle \text{statement} \rangle$, and we would like to derive a contradiction. To this end, we define an $\langle \text{type} \rangle$ using $\langle \text{term} \rangle$. We then show that such an $\langle \text{type} \rangle$ is closely associated with a $\langle \text{type} \rangle$. Finally, we derive a contradiction by showing that $\langle \text{statement} \rangle$.

In this chapter we give a further explanation of the above diagram. As we can easily see, $\langle \text{type} \rangle$ and $\langle \text{type} \rangle$ play leading roles in the proof. By following the outline of the proof, the reader should familiarize her/himself with these two subjects. We indicate where the details of certain topics are treated in the main text. Skipping some unfamiliar terminology, the reader should grasp the flow of the proof

a marked up proof text before LLMs

THEOREM 0.1. *Let n be an integer greater than or equal to 3. If integers X , Y , and Z satisfy the equation*

$$(0.1) \quad X^n + Y^n = Z^n,$$

then at least one of X , Y , and Z must be 0.

A flow diagram of the proof can be drawn as follows:

$$(0.2) \quad \begin{array}{ll} (\text{a solution of (0.1)}) & \implies (\text{an elliptic curve}) \\ & \implies (\text{a modular form}) \\ & \implies (\text{contradiction}) \end{array}$$

If we try to explain the meaning of this diagram in a few sentences, it goes as follows. Assume there exists a nontrivial solution to the equation (0.1), and we would like to derive a contradiction. To this end, we define an elliptic curve using such a solution. We then show that such an elliptic curve is closely associated with a modular form with certain properties. Finally, we derive a contradiction by showing that such a modular form could not exist.

In this chapter we give a further explanation of the above diagram. As we can easily see, elliptic curves and modular forms play leading roles in the proof. By following the outline of the proof, the reader should familiarize her/himself with these two subjects. We indicate

the approach: a pipeline

scanned mathematical source
(article/textbook)

⋈

L^AT_EX clone of the source

⊃

massaged into a blueprint

⋈

formal proof sketch of the blueprint

⊃

full formalization

⋈ as close as possible

⊃ should not omit anything

the approach: an improved pipeline

scanned mathematical source
(article/textbook)

⋈

L^AT_EX clone of the source

⋈

massaged into a blueprint

⊍

sentence parts annotated in the blueprint

⋈

formal proof sketch of the blueprint

⊍

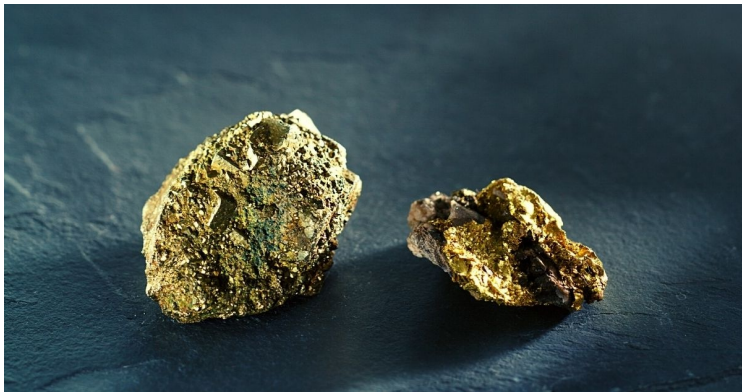
full formalization

⋈ as close as possible

⊍ should not omit anything

suggestions?

do LLMs give us mostly real gold or mostly fool's gold?



do you integrate AI tools into your workflow?

There is a monster in the forest and it speaks with a thousand voices. It will answer any question you pose it, it will offer insight to any idea. It will help you, it will thank you, it will never bid you leave. It will even tell you of the darkest arts, if you know precisely how to ask.

It feels no joy and no sorrow, it knows no right and no wrong. It knows not truth from lie, though it speaks them all the same.

It offers its services freely to any passerby, and many will tell you they find great value in its conversation. 'You simply must visit the monster – I always just ask the monster.'

There are those who know these forests well; they will tell you that freely offered doesn't mean it has no price.

For when the next traveler passes by, the monster speaks with a thousand and one voices. And when you dream you see the monster; the monster wears your face.

do you integrate AI tools into your workflow?

There is a monster in the forest and it speaks with a thousand voices. It will answer any question you pose it, it will offer insight to any idea. It will help you, it will thank you, it will never bid you leave. It will even tell you of the darkest arts, if you know precisely how to ask.

It feels no joy and no sorrow, it knows no right and no wrong. It knows not truth from lie, though it speaks them all the same.

It offers its services freely to any passerby, and many will tell you they find great value in its conversation. 'You simply must visit the monster – I always just ask the monster.'

There are those who know these forests well; they will tell you that freely offered doesn't mean it has no price.

For when the next traveler passes by, the monster speaks with a thousand and one voices. And when you dream you see the monster; the monster wears your face.

table of contents

contents

opinions on AI

autoformalization

the project

suggestions?

table of contents